

Universidad Autónoma de Baja California

Proyecto de Carrera

Taller 8 – Diseño de la arquitectura integral de HydroMate

Valdez Meza Kevin Osmar
1288703

14 – Octubre – 2026

1. Propósito y decisiones de arquitectura

HydroMate monitorea y controla una torre hidropónica recirculante. Este documento fija la arquitectura con la que se continuará la implementación: sensores y control local en ESP32, comunicación MQTT/TLS, backend Laravel, PostgreSQL y aplicación Android por HTTPS.

La revisión del 21 de octubre conserva el formato y la base del taller del 14 de octubre. El flujo público de telemetría ya está integrado, incluido TDS en A1. Autenticación de usuarios, notificaciones, aprovisionamiento BLE y control de actuadores se presentan como decisiones de diseño; su selección no afirma que estén implementados o probados.

Decisión	Justificación para HydroMate
ESP32 con seguridad local	Nivel bajo, sensores inválidos y límites deben vetar la actuación sin depender de Internet.
Conservar Arduino-ESP32 y modularizar	Reutiliza el firmware existente. FreeRTOS permite separar seguridad, adquisición y red dentro del mismo entorno [R1].
Mosquitto y MQTT/TLS	Mantiene tópicos, cuentas y bridge existentes; separa publicación de sensores y consumo de datos [R2].
Laravel y PostgreSQL	Una API concentra validación, acceso por torre e historial. Se reutilizan backend y esquema ya integrados.
Android nativo Kotlin por HTTPS	Conserva las pantallas y el repositorio HTTP; separa presentación y datos sin cambiar la tecnología de la app [R3].
Sanctum para usuarios	Tokens revocables para Android y autorización por dispositivo dentro de Laravel; OAuth2 completo no es necesario para este alcance [R4].
Laravel + FCM para notificaciones	Laravel decide y guarda la alerta; FCM entrega el aviso al teléfono. Evita exigir consultas continuas en segundo plano [R5].
BLE para alta inicial de Wi-Fi	Permite configurar la red desde Android cerca del equipo. Se limita a una sesión de aprovisionamiento segura; no controla actuadores [R6].

Estas elecciones son adecuadas al hardware disponible y al software construido. Los límites de dosificación y el mecanismo de cada canal se determinan mediante validación física; no se inventan valores para cerrar el diseño.

2. Arquitectura general y despliegue

La torre y el teléfono se conectan por Internet a la VPS. Los servicios de HydroMate se ejecutan bajo deploy-hydromate con Docker rootless, redes y volúmenes propios. La PC conserva únicamente el entorno de desarrollo [R7].

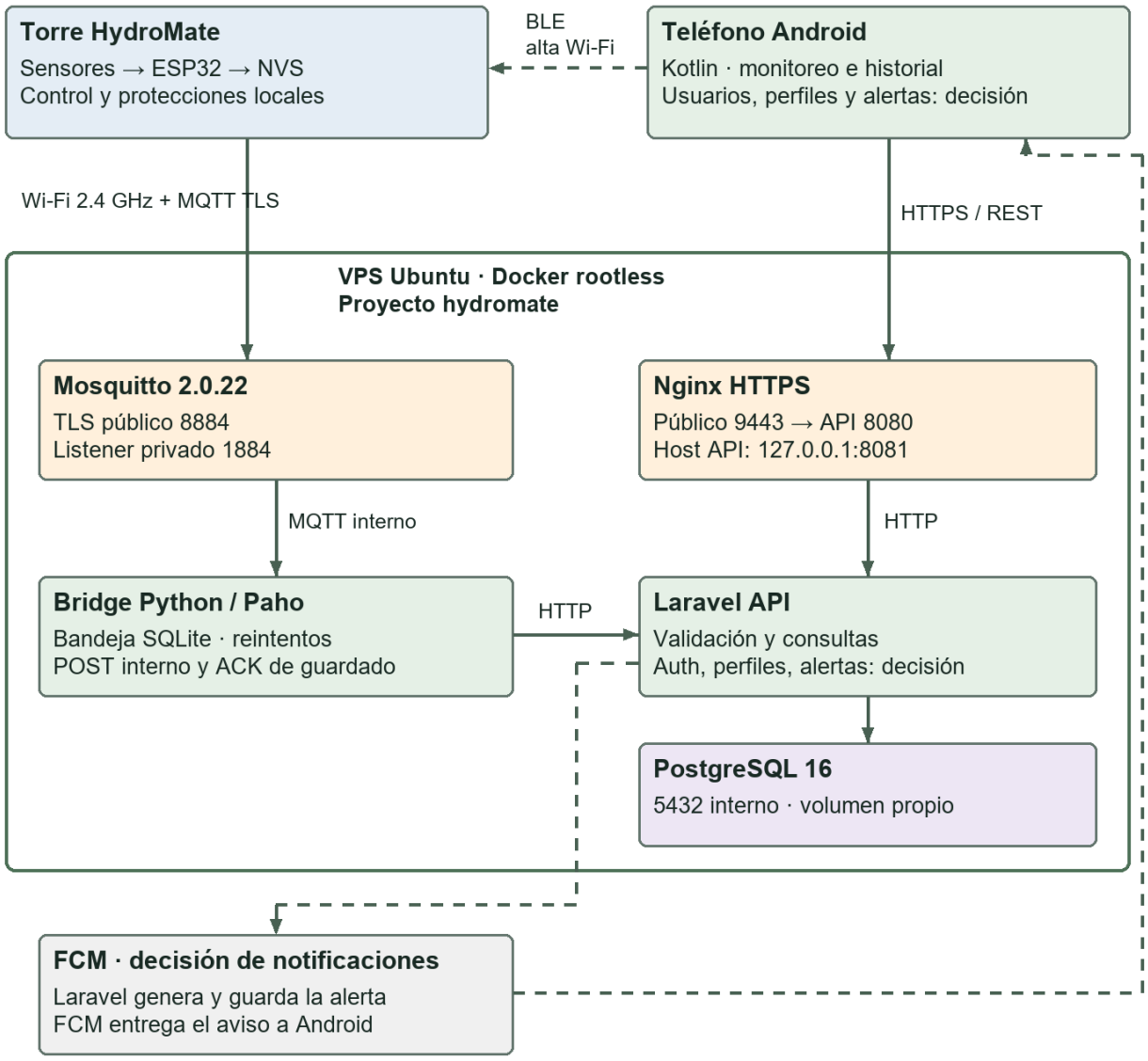


Figura 1. Arquitectura y ubicación de los componentes. PostgreSQL solo es accesible desde Laravel.

API pública: https://hydromate-kevin.duckdns.org:9443. MQTT TLS: hydromate-kevin.duckdns.org:8884. El retorno ACK confirma guardado; Android consulta la API y no abre conexiones a la base de datos.

3. Hardware y alimentación

El montaje de adquisición utiliza DS18B20, BH1750, flotador y módulos analógicos de pH y TDS mediante ADS1115. Se conservan los pines comprobados del prototipo. El bloque de actuación define las conexiones que deberá implementar el controlador.

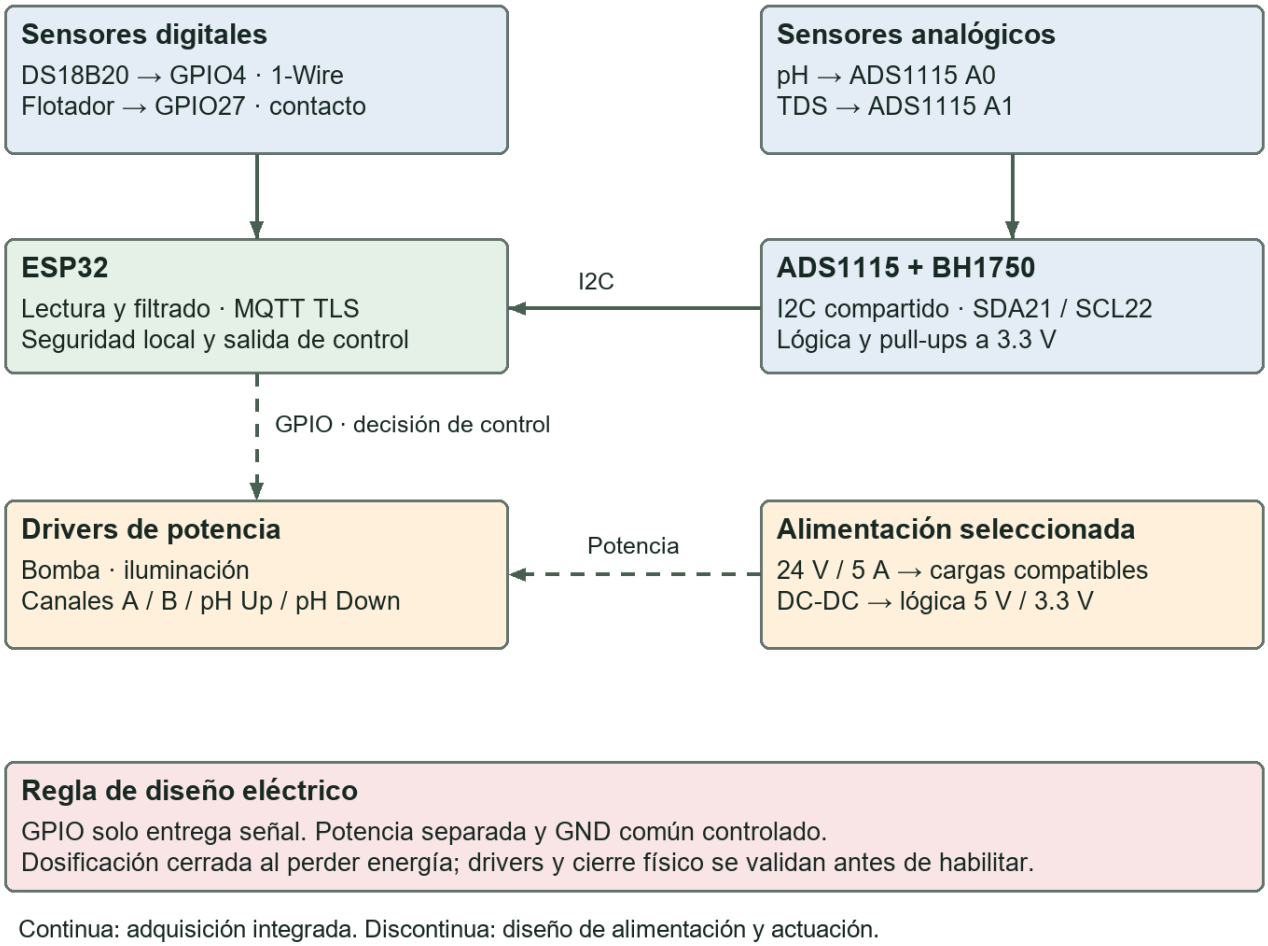


Figura 2. Separación entre sensores, lógica, drivers y alimentación. Las líneas de potencia no representan conexiones directas a GPIO.

Decisión eléctrica: lógica e I2C a 3.3 V; alimentación de cargas por una rama separada. Con ADS1115 alimentado a 3.3 V, las entradas analógicas deben permanecer entre GND y VDD. La escala ±6.144 V del PGA no permite superar VDD [R8]. La salida del módulo pH requiere acondicionamiento si puede exceder ese límite.

La fuente seleccionada es de 24 V / 5 A. La bomba de 24 V utiliza driver D4184 compatible con su corriente y señal; los convertidores DC-DC alimentan la lógica. Cada canal A, B, pH Up y pH Down mantiene una interfaz de driver independiente y cierre sin energía. El mecanismo exacto se valida antes de conectar la carga.

4. Tabla de entradas salidas y pines

E y S se definen respecto del ESP32. Las entradas de sensores conservan el montaje; las salidas indicadas como asignación preliminar son parte del diseño y no una autorización para energizar actuadores.

Elemento	E/S y conexión	Voltaje o señal	Uso
DS18B20	E/S · GPIO4 · 1-Wire	3.3 V; pull-up 4.7 kΩ	Temperatura del agua, °C.
Flotador	E · GPIO27 a GND	INPUT_PULLUP 3.3 V	LOW: agua; HIGH: nivel bajo/cable abierto.
Bus I2C	E/S · SDA21 / SCL22	3.3 V y GND común	Bus compartido de ADS1115 y BH1750.
ADS1115	E/S · I2C 0x48	VDD 3.3 V	Digitaliza pH y TDS; ADDR a GND.
pH PH-4502C	E · ADS1115 A0	Módulo 5 V; señal ≤VDD del ADS	pH sin unidad; acondicionar señal.
TDS analógico	E · ADS1115 A1	Señal dentro de 0–3.3 V	ppm; compensación térmica con DS18B20.
BH1750	E · I2C compartido	3.3 V; dirección según ADDR	Iluminación en lux, no porcentaje ni PAR.
Bomba de recirculación	S · GPIO25 preliminar	Lógica 3.3 V → driver → 24 V	Conmutación segura de bomba.
Iluminación	S · GPIO26 preliminar	Lógica 3.3 V → driver compatible	Alimentación según lámpara seleccionada.
Nutrientes A y B	S · GPIO13 / 14 preliminares	Señal 3.3 V; potencia por driver	Dos canales independientes.
pH Up y pH Down	S · GPIO18 / 19 preliminares	Señal 3.3 V; potencia por driver	Canales mutuamente excluyentes.
Alarma local	S · GPIO23 preliminar	3.3 V hacia driver si requiere corriente	Aviso local según configuración.
Habilitación TDS	S · GPIO32 preliminar	3.3 V → interruptor high-side	Decisión de aislamiento durante adquisición química.

A2 y A3 del ADS1115 quedan sin asignación. Se reservan pines de flash, arranque y UART de programación; la variante de placa y los niveles durante reset se verifican antes de fabricar. Los GPIO solo entregan señales. INA219/INA226 y medición de consumo no forman parte del mínimo de esta arquitectura.

5. Módulos de software y almacenamiento

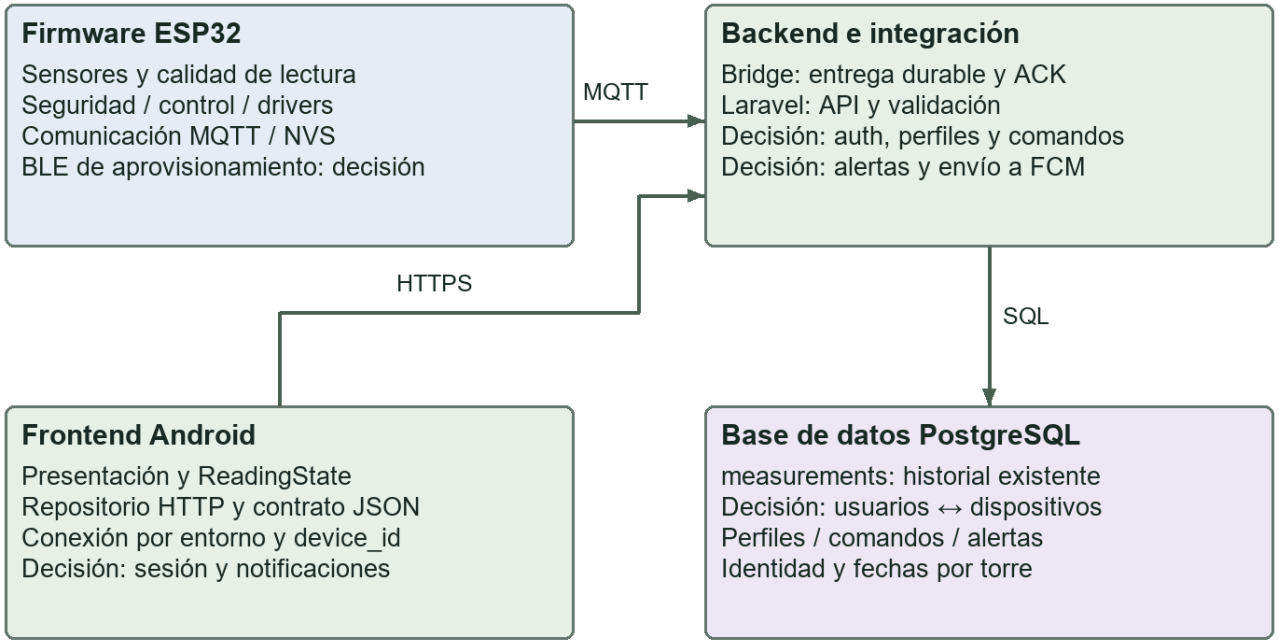


Figura 3. Módulos por responsabilidad. La comunicación y persistencia se separan de las decisiones de seguridad.

Módulo	Responsabilidad y datos que maneja
Sensores y adquisición · ESP32	Lee, filtra y convierte temperatura, pH, TDS, nivel y luz; produce lectura con calidad y fecha.
Seguridad y control · ESP32	Decisión: evalúa nivel, validez, dosis y tiempos; entrega permiso o veto a drivers de bomba, luz y cuatro canales.
Comunicación y NVS · ESP32	Publica telemetría, conserva secuencia/pendiente y procesa ACK; separa reconexión de control local.
Bridge Python	Recibe MQTT, conserva bandeja SQLite, entrega a Laravel y confirma persistencia. No escribe directamente en PostgreSQL.
API Laravel	Valida y guarda mediciones; sirve consultas. Decisión: usuarios, perfiles, configuración, comandos y alertas por torre.
PostgreSQL	Persiste measurements con id y UNIQUE(device_id, sequence). Decisión: relacionar usuarios/dispositivos, perfiles, comandos y alertas.
Android	Presenta lectura, fecha, historial y errores; repositorio HTTP y configuración de entorno. Decisión: sesión, BLE y recepción FCM.

Decisión del modelo de datos: cada medición pertenece a una torre; una cuenta puede acceder a varias torres mediante una relación de permisos. Perfiles, comandos y alertas conservan device_id y fecha. Las nuevas entidades se incorporan con migraciones aditivas, sin borrar el historial existente.

6. Flujos de datos y protocolos

Telemetría y consulta

1. El ESP32 lee y filtra sensores; asigna device_id, sequence y captured_at cuando su reloj está sincronizado. Guarda una muestra pendiente en NVS y publica el mismo JSON hasta recibir confirmación.

2. Mosquitto verifica la cuenta y ACL. El bridge recibe el mensaje, conserva la bandeja SQLite y lo entrega por POST interno a Laravel. Laravel valida y registra en PostgreSQL; el bridge publica ACK stored después de comprobar el guardado.

3. Android consulta última medición e historial por HTTPS con device_id y real_only=1. Muestra adquisición/recepción y antigüedad; no reemplaza datos ausentes por valores inventados.

Adquisición solicitada control y alertas

El botón de actualización solicita una medición con UUID; el bridge publica commands/measure y el ESP32 responde por telemetría v3 correlacionada. La solicitud solo se satisface al guardar la nueva adquisición dentro de su vigencia.

Decisión de control: Android autenticado → Laravel autoriza por torre → orden MQTT con ID y caducidad → ESP32 valida y aplica su veto local → resultado y registro. Decisión de alertas: Laravel evalúa límites, guarda el evento y envía el aviso mediante FCM; Android consulta el detalle autorizado.

Enlace	Protocolo	Puerto o canal
Sensores ↔ ESP32	1-Wire, contacto, I2C y ADC	GPIO4/27; SDA21/SCL22; ADS A0/A1.
ESP32 ↔ red	Wi-Fi 2.4 GHz	Acceso IP a Internet.
ESP32 ↔ Mosquitto	MQTT 3.1.1 sobre TLS	8884 público; CA y hostname validados.
Bridge ↔ Mosquitto / Laravel	MQTT / HTTP internos	1884 y API 8080 en red Docker privada.
Android ↔ Laravel	HTTPS/REST · JSON	9443 público; API host 8081 solo loopback.
Laravel ↔ PostgreSQL	SQL mediante controlador PostgreSQL	5432 interno, sin publicación al host.
Alta Wi-Fi / avisos	BLE GATT / FCM HTTP v1	Decisiones: sesión local BLE; envío FCM desde Laravel.

7. Formato de telemetría

Se conserva el contrato JSON UTF-8 v3. Este ejemplo es ilustrativo del formato, no un resultado de calibración ni una nueva medición física.

```
{
  "message_version": 3,
  "device_id": "hydromate-01",
  "sequence": 4800,
  "reason": ["periodic_poc"],
  "captured_at": "2026-10-21T09:00:00Z",
  "temperature_c": 23.5, "light_lux": 265.0,
  "water_present": true, "ph": 6.2,
  "tds_ppm": 770.0, "ec_us_cm": null,
  "sources": {
    "temperature_c": "real", "light_lux": "real",
    "water_present": "real", "ph": "real",
    "tds_ppm": "real", "ec_us_cm": "unavailable"
  }
}
```

Campo o regla	Definición
Identidad	device_id estable y sequence entero positivo. La pareja identifica una adquisición y evita duplicados.
Fecha	captured_at UTC cuando existe reloj válido; si no, null. Laravel asigna created_at al recibir.
Variables y unidades	temperature_c: °C; light_lux: lux; ph: sin unidad; tds_ppm: ppm; ec_us_cm: µS/cm; water_present: booleano.
Calidad por variable	sources: real, simulated, unavailable o invalid. Los dos últimos exigen valor null; false del flotador es válido.
Ausencias	EC permanece null/unavailable sin sensor independiente. No convertir TDS en EC ni usar cero para un sensor ausente.
TDS y temperatura	Compensar con DS18B20 válido. Sin temperatura válida, referencia de 25 °C señalada en Android como sin compensación térmica.
Límites actuales	pH 0–14; temperatura 0–50 °C; luz 0–100000 lux; TDS 0–1000 ppm; EC 0–100000 µS/cm. Son límites del contrato, no del cultivo.

Los límites agronómicos se guardan en perfiles autorizados y se verifican independientemente de los rangos admitidos por la API. La compatibilidad con mensajes v1/v2 se conserva sin duplicar sus ejemplos en este taller.

8. Endpoints principales de la API

Las rutas existentes sostienen el flujo público integrado. Las filas marcadas como decisión definen las operaciones del diseño; no afirman endpoints ya programados. Toda ruta del dominio debe aplicar autorización por torre al incorporar usuarios.

Método y ruta	Entrada o salida principal	Condición
POST /api/measurements	Recibe v3; 201 registrado, 422 inválido, 409 duplicado.	Existente; acceso interno del bridge.
GET /api/measurements	device_id, limit y real_only; devuelve historial.	Existente; filtra identidad y origen.
GET /api/measurements/latest	device_id; devuelve última medición.	Existente; 404 si no hay datos.
GET /api/devices/{id}/status	no_data, recent o stale y fechas.	Existente; antigüedad de telemetría.
POST /api/devices/{id}/measurement-requests	Cuerpo vacío; devuelve UUID de solicitud.	Existente; limitado a 12/min y torre conocida.
GET /api/measurement-requests/{uuid}	Estado de adquisición solicitada.	Existente; vigencia y correlación.
GET /api/internal/measurement-requests	Solicitudes para el bridge.	Existente; solo red interna.
POST /api/auth/login; DELETE /api/auth/session	Credenciales → token; revocar sesión.	Decisión Sanctum.
GET /api/devices; GET/PATCH /api/devices/{id}/configuration	Torres autorizadas y configuración versionada.	Decisión de acceso y configuración.
GET/POST /api/crop-profiles	Consultar o guardar perfiles y sus versiones.	Decisión de perfiles de cultivo.
POST /api/devices/{id}/commands	Orden con parámetros e ID; respuesta de aceptación.	Decisión; no equivale a actuación.
GET /api/devices/{id}/alerts; POST /api/notification-tokens	Alertas por torre; registrar destino FCM del usuario.	Decisión de notificaciones.

Nginx permite las consultas públicas y el POST acotado de adquisición; la inserción de mediciones permanece interna. La API conserva respuestas comprensibles y errores por campo. No es necesario enumerar todos los códigos de error para congelar esta arquitectura.

9. Tópicos eventos y comandos

Se conserva el prefijo hydromate/{id}. No se crea un árbol de tópicos nuevo por cambiar la versión del mensaje. Las mediciones y órdenes no se publican como retained para evitar interpretarlas como adquisiciones u órdenes recientes.

Tópico	Emisor → receptor	Mensaje y función
hydromate/{id}/telemetry	ESP32 → bridge	Existente: medición JSON v3; publicación de la placa QoS 0.
hydromate/{id}/ack	Bridge → ESP32	Existente: sequence + status; stored, rejected o conflict.
hydromate/{id}/commands/measure	Bridge → ESP32	Existente: action, request_id y expires_at; QoS 1, sin retained.
hydromate/{id}/commands	Laravel/publicador → ESP32	Decisión: orden de control con UUID, acción, parámetros y caducidad.
hydromate/{id}/command_acks	ESP32 → integración	Decisión: recepción, rechazo y resultado de una orden.
hydromate/{id}/events	ESP32 → integración	Decisión: nivel bajo, sensor inválido, actuación y fallo; evento con ID y fecha.

Confirmación de persistencia y solicitud de medición

```
ACK: {"sequence":4800,"status":"stored"}
Solicitud:
{"action":"measure","request_id":"<UUID>",
 "expires_at":1792576800}
```

El ACK de aplicación confirma la persistencia en PostgreSQL y es distinto del ACK de transporte MQTT. La placa conserva el mismo pendiente hasta stored. Ante 409, el bridge compara el contenido existente antes de confirmar; si difiere, informa conflict.

Decisión del formato de control

```
{"schema_version":"hm-control-1","type":"command",
 "message_id":"<UUID>","device_id":"hydromate-01",
 "sequence":1,"captured_at":"2026-10-21T09:00:00Z",
 "payload":{"command_id":"<UUID>",
   "action":"irrigation.stop","parameters":{},
   "expires_at":"2026-10-21T09:00:30Z","status":"pending"}}
```

hm-control-1 se reserva para actuación y eventos; no sustituye la telemetría v3 ni el comando measure existente. El ESP32 valida identidad, acción, duplicados y vigencia. Una orden aceptada por la API solo indica que fue registrada; el resultado físico requiere confirmación separada.

10. Autenticación y protección de credenciales

Usuarios dispositivos y permisos

Decisión de usuarios: Laravel Sanctum emite tokens revocables para Android y las Policies verifican el acceso a cada torre. Se elige sobre un servidor OAuth2 completo porque la app es un cliente propio y ya existe Laravel [R4]. Los perfiles públicos o compartidos no conceden control sobre una torre.

Decisión de dispositivos: cada torre utiliza una cuenta MQTT y ACL restringida a sus tópicos. El bridge utiliza otra cuenta. TLS valida CA y hostname; se prohíbe setInsecure. device_id y sources identifican el mensaje, pero no sustituyen autenticación.

Alcance actual: MQTT ya autentica cuentas y aplica ACL. La API pública de consulta y adquisición todavía no implementa el modelo de usuarios descrito arriba; ese acceso no se presenta como privacidad por cuenta.

Información	Almacenamiento y protección elegida
Wi-Fi y MQTT del ESP32	secrets.h privado y excluido de Git en el banco. Decisión de producto: alta BLE con sesión segura y prueba de posesión; configuración persistente y borrado controlado.
APP_KEY, BD y credenciales del bridge	/etc/hydromate/.env únicamente en VPS; permisos mínimos y fuera del contexto de build y del repositorio.
Certificados y claves privadas	/etc/hydromate/https y /etc/hydromate/mqtt; mounts de solo lectura y permisos del usuario interno del servicio.
Token de sesión Android	Decisión: cifrado mediante clave protegida por Android Keystore; no guardarlo en preferencias de conexión ni imprimirlo en logs.
Credencial de envío FCM	Decisión: credencial del servidor fuera de Git y APK. Laravel asocia el token de registro FCM con el usuario; ese token no concede acceso a la API.
Contraseñas y permisos	Hash de contraseña, revocación y permisos por torre; limitar frecuencia, tamaño de mensajes y acciones aceptadas. Nunca incluir secretos en telemetría.

Decisión de notificación: Laravel guarda y deduplica la eventualidad, luego envía por FCM HTTP v1. Android requiere permiso de notificaciones y servicios Google compatibles. FCM transporta el aviso, no reemplaza Laravel/PostgreSQL ni garantiza una parada de seguridad [R5].

11. Comportamiento sin Internet y tras reinicio

La recuperación separa entrega de datos y seguridad física. El firmware actual conserva secuencia y una muestra pendiente en NVS. Las reglas de control de esta tabla son decisiones para el controlador, no resultados de pruebas de actuadores.

Situación	Comportamiento definido
Wi-Fi, Internet o broker caído	Adquisición y control local independientes de red. Conservar la muestra pendiente y reconectar con espera; no aceptar órdenes nuevas sin conexión.
Cambios durante una desconexión	Entregar primero la adquisición pendiente y después el estado reciente. El mecanismo actual no garantiza conservar cada transición intermedia.
Laravel o PostgreSQL caído	El bridge conserva su bandeja SQLite y reintenta; no publica stored antes de verificar persistencia. PostgreSQL conserva los registros confirmados.
Reinicio del ESP32	Recuperar secuencias y pendiente NVS; reenviar con la misma identidad. Decisión de actuación: salidas seguras y dosis cerradas antes de habilitar control.
Vuelve la energía	Validar configuración, sensores, nivel y límites. No reanudar una dosis interrumpida ni ejecutar una orden caducada. Sin UPS, no hay operación durante el corte.
Reloj sin sincronizar	Telemetría captured_at=null y hora de recepción del backend. Temporizadores de seguridad monotónicos; rechazar activación remota cuya vigencia no pueda verificarse.
Sensor inválido o nivel bajo	Enviar null con origen invalid y conservar las demás variables. Decisión: bloquear la actuación dependiente; nivel bajo impide bomba y dosificación.
Android sin red	Mostrar error o lectura antigua con su fecha. Nunca mostrar datos demo como producción; las notificaciones requieren conectividad y permisos del teléfono.
Servicios Docker reiniciados	restart unless-stopped y volúmenes propios. Healthchecks detectan fallos; la supervisión debe atender estados unhealthy y comprobar recuperación.

Para el control químico se decide medir → validar → dosificar un solo canal → cerrar → mezclar → esperar → volver a medir. Se excluyen pH Up/Down simultáneos y dosis sin calibración o límites válidos. El tiempo máximo de cada acción no puede ampliarse por una espera de red.

No se utiliza deep sleep durante la operación normal del controlador. Se priorizan supervisión continua y recepción de órdenes; las estrategias de ahorro no pueden suspender los interbloqueos.

12. Tecnologías y versiones principales

Se conservan las tecnologías del repositorio y se fijan las decisiones necesarias para completar el sistema. No se exige fijar aquí cada biblioteca secundaria; las versiones reproducibles se mantienen en los archivos de dependencias y las imágenes de despliegue.

Tecnología	Versión o selección	Uso
ESP32	DevKit de ESP32 clásico	Adquisición, control y conectividad Wi-Fi/BLE.
Arduino-ESP32 / C++	Core 3.3.11 registrado	Conservar entorno actual; modularizar y usar tareas FreeRTOS cuando corresponda [R1].
NVS / Preferences	Incluida en Arduino-ESP32	Secuencias, muestra pendiente y configuración persistente [R9].
Mosquitto / MQTT	2.0.22 / MQTT 3.1.1	Broker propio, cuentas ACL y MQTT/TLS 8884.
Bridge Python / Paho	Python 3 / Paho 2.1.0	Integración MQTT→API, SQLite de bandeja y ACK.
Laravel / PHP	Laravel 13.33.0 en lock; PHP 8.4 en imagen	API, validación y módulos de dominio dentro de un backend.
PostgreSQL	16	Persistencia relacional e identidad por torre.
Android / Kotlin	App 0.7.0; minSdk 26, target 35, compile 37	Cliente nativo existente; separación presentación/repositorio.
Nginx / Docker Compose	nginx-unprivileged; Docker rootless	HTTPS 9443 y servicios aislados del proyecto hydromate.
Sanctum	Decisión: rama4.x compatible con Laravel 13	Autenticación mediante tokens y autorización por torre.
Firebase Cloud Messaging	Decisión: API HTTP v1 y SDKAndroid compatible	Avisos remotos; backend emisor permanece en la VPS.
BLE / TLS / JSON	GATT; TLS 1.2 o superior; JSON UTF-8	Alta segura, transporte cifrado y contratos interoperables.

Criterio de selección

La elección conserva componentes ya integrados y evita una migración de framework para cumplir el taller. Sanctum resuelve el acceso de una app propia; FCM atiende la entrega en segundo plano; BLE resuelve el alta local. Estas conclusiones son decisiones de proyecto sustentadas en la documentación oficial, no una afirmación de que sean la única alternativa posible.

13. Trazabilidad y congelamiento

Requisito del taller	Componente o documento	Ubicación
Diagrama general de arquitectura	ESP32, broker, backend, BD, Android y FCM	§2 · figura 1
Diagrama de despliegue	VPS rootless y fronteras pública/interna	§2 · figura 1
Diagrama de hardware	Sensores, buses, drivers y alimentación	§3 · figura 2
Módulos de firmware/backend/BD/frontend	Agrupación de software y almacenamiento	§5 · figura 3
Responsabilidad de cada módulo	Tabla de módulos y datos que manejan	§5
Flujo entre componentes	Telemetría, adquisición, control y alerta	§6
Tabla de entradas y salidas	Sensores y salidas del ESP32	§4
Pines y niveles de voltaje	GPIO/bus/ADC y separación lógica/potencia	§3–4
Protocolos de comunicación	Wi-Fi, MQTT/TLS, REST, SQL y BLE	§6
Formato preliminar de mensajes	JSONv3, ACK, measure y control	§7 y9
Endpoints tópicos y eventos	Operaciones existentes y decisiones de interfaz	§8–9
Autenticación de usuarios/dispositivos	Sanctum/Policies y cuentas MQTT/ACL	§10
Credenciales y variables de entorno	Archivos privados, TLS y almacén Android	§10
Sin Internet y después de reinicio	NVS, bandeja durable y estado seguro	§11
Tecnologías y versiones principales	Stack conservado y servicios seleccionados	§12
Matriz requisito → componente	Esta matriz relaciona todos los mínimos	§13

Regla de cambios

Esta entrega congela la arquitectura principal: ESP32 con seguridad local; Mosquitto MQTT/TLS; bridge de integración; Laravel; PostgreSQL; Android HTTPS. Auth, alertas y aprovisionamiento quedan asignados a Sanctum, FCM y BLE respectivamente como decisiones de diseño.

Todo cambio posterior se registra en la bitácora con fecha, motivo técnico, decisión sustituida, componentes/contratos afectados y verificación necesaria. La decisión anterior se conserva como antecedente. Ajustar pines, parámetros o mecanismos exige justificar el cambio y comprobar que se mantienen los límites de seguridad.

14. Referencias y base técnica

Fuentes oficiales consultadas para justificar las decisiones, 21 de octubre de 2026. Las referencias de fabricante establecen capacidades y límites; no equivalen a una calibración o prueba del montaje.

[R1] Espressif. Ejemplo oficial de tareas FreeRTOS en Arduino-ESP32.

https://github.com/espressif/arduino-esp32/blob/master/libraries/ESP32/examples/FreeRTOS/BasicMultiThreading/BasicMultiThreading.ino

[R2] Eclipse Mosquitto 2.0.22. Configuración TLS, autenticación y ACL.

https://github.com/eclipse-mosquitto/mosquitto/blob/v2.0.22/man/mosquitto.conf.5.xml

[R3] Android Developers. Separación de presentación y datos.

https://developer.android.com/topic/architecture

[R4] Laravel 13. Sanctum y autenticación de aplicaciones móviles.

https://laravel.com/docs/13.x/sanctum

[R5] Firebase. Servidor emisor FCM y cliente Android.

https://firebase.google.com/docs/cloud-messaging/server-environment

[R5a] Firebase. Permisos, SDK y compatibilidad de Android.

https://firebase.google.com/docs/cloud-messaging/android/get-started

[R6] Espressif. Aprovisionamiento Wi-Fi mediante BLE y sesiones seguras. Referencia de diseño; no obliga a cambiar el SDK del firmware.

https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32/api-reference/provisioning/wifi_provisioning.html

[R7] Docker. Daemon y contenedores en modo rootless.

https://docs.docker.com/engine/security/rootless/

[R8] Texas Instruments. ADS111x, hoja de datos SBAS444E, límites de entrada y alimentación.

https://www.ti.com/lit/ds/symlink/ads1115.pdf

[R9] Espressif. Preferences y persistencia NVS.

https://docs.espressif.com/projects/arduino-esp32/en/latest/api/preferences.html

Documentación del proyecto

T6 actualizado: requisitos y alcance. docs/ARCHITECTURE.md y docs/DECISIONS.md: decisiones e historial. docs/TELEMETRY_CONTRACT.md y docs/API.md: contratos ejecutables. compose.production.yml: despliegue. embedded/, scripts/, hydromate-backend/ y mobile/: implementación.

El reporte de pruebas y las evidencias físicas se conservan en el Taller 5. Este documento se concentra en el diseño, las responsabilidades y los contratos necesarios para continuar la implementación.